

Capítulo 17

Classes: uma visão mais detalhada, parte 1

Exercícios — Resolução Didática com Código C++

“Neste documento resolvemos os 11 exercícios do Capítulo 17, sendo o primeiro conceitual e os dez restantes de programação. Os códigos seguem o padrão estabelecido no Capítulo 16 — um trio (`.h`, `.cpp`, `main`) por classe — e exploram quatro temas centrais: enriquecimento de classes existentes (`Time`, `Date`), definição de novas classes matemáticas (`Complex`, `Rational`, `HugeInteger`), abstração de figuras geométricas (`Retangulo` em três níveis de sofisticação) e modelagem de um jogo completo (`JogoVelha`). Todos os códigos compilam com `g++ -Wall -Wextra -std=c++14` sem warnings.”

1 Exercício 17.3 — Finalidade do operador de resolução de escopo

Enunciado. Qual é a finalidade do operador de resolução de escopo?

Resposta

O operador de resolução de escopo `::` (lê-se “dois-pontos-dois-pontos” ou simplesmente “escopo”) tem duas formas em C++, já antecipadas no Capítulo 15 (unária) e introduzidas no Capítulo 16 (binária):

- **Forma unária** (`::nome`): acessa um identificador no *escopo global*, quando ele está sendo ocultado por uma variável local de mesmo nome.
- **Forma binária** (`Classe::membro` ou `Namespace::membro`): especifica explicitamente a qual *escopo nomeado* o identificador pertence.

Três usos típicos da forma binária

1. **Definir funções-membro fora da classe.** Esta é a aplicação mais central deste capítulo. No `.h` declaramos os protótipos; no `.cpp` definimos os corpos, qualificando cada um:

```
// No Time.h:
class Time {
public:
    void setHour(int h);
private:
    int hour;
};

// No Time.cpp:
void Time::setHour(int h) { hour = h; }
// ~~~~~ Sem este "Time::", a funcao seria considerada
// uma funcao livre, nao um metodo de Time.
```

2. **Acessar membros estáticos.** Membros declarados `static` pertencem à classe, não a um objeto específico. Acessam-se assim: `Math::PI`, `Conta::numeroDeContas()`.
3. **Qualificar nomes em *namespaces*.** Veremos cotidianamente: `std::cout`, `std::vector`, `std::string`. Aqui `std` é o *namespace*, não uma classe.

Por que essa sintaxe é essencial?

Observação de Engenharia de Software

Sem o operador `::`, a separação entre arquivo de cabeçalho e arquivo de implementação seria impraticável. O compilador precisa de algum jeito de *amarrar* a definição livre em `Time.cpp` à declaração dentro da classe em `Time.h`. Esse jeito é o prefixo `Time::`. Como subproduto, isso permite que você tenha duas classes diferentes com funções-membro de mesmo nome — `Conta::saldo()` e `Investimento::saldo()` convivem harmoniosamente.

Boa Prática de Programação

Procure sempre definir as funções-membro fora da definição da classe, no arquivo `.cpp`. A definição *dentro* da `class` torna o cabeçalho extenso, oculta a interface entre detalhes de implementação e força a recompilação de todos os clientes quando você altera o corpo da função. A exceção são funções muito pequenas que se beneficiam de *inlining* explícito (Cap. 15).

2 Exercício 17.4 — Time com hora corrente do sistema

Enunciado. Forneça um construtor capaz de usar a hora atual das funções `time` e `localtime` (declaradas em `<ctime>`) para inicializar um objeto `Time`.

Análise do problema

A biblioteca C `<ctime>` (renomeada de `<time.h>` de C) oferece duas funções-chave:

- `time_t time(time_t *)`: retorna o tempo atual em segundos desde a *epoch* (1/1/1970 UTC).
- `struct tm *localtime(const time_t *)`: converte esse número em uma estrutura com campos `tm_hour`, `tm_min`, `tm_sec` (e mais campos para data, fuso, etc.) já no fuso horário local.

Adotaremos um construtor com um parâmetro booleano (`useCurrentTime`) que decide se usa a hora corrente ou os valores fornecidos.

Cabeçalho `Time.h`

```

1 // Exercício 17.4 - Time.h
2 // Classe Time com construtor que pode usar a hora corrente do sistema
3 #ifndef TIME_H
4 #define TIME_H
5
6 class Time {
7 public:
8     // Construtor: se useCurrentTime=true, usa a hora atual do sistema;
9     //                caso contrario, usa os valores fornecidos (default 0,0,0)
10    explicit Time(bool useCurrentTime = false,
11                  int h = 0, int m = 0, int s = 0);

```

```

12
13     void setTime(int h, int m, int s);
14     void setHour(int h);
15     void setMinute(int m);
16     void setSecond(int s);
17
18     int getHour() const;
19     int getMinute() const;
20     int getSecond() const;
21
22     void printUniversal() const;    // formato 24h: HH:MM:SS
23     void printStandard() const;    // formato 12h: HH:MM:SS AM/PM
24
25 private:
26     int hour;        // 0-23
27     int minute;      // 0-59
28     int second;      // 0-59
29 };
30
31 #endif

```

Implementação Time.cpp

```

1  // Exercício 17.4 - Time.cpp
2  #include <iostream>
3  #include <iomanip>
4  #include <ctime>
5  #include "Time.h"
6  using namespace std;
7
8  Time::Time(bool useCurrentTime, int h, int m, int s) {
9      if (useCurrentTime) {
10         time_t agora = time(nullptr);
11         struct tm *partes = localtime(&agora);
12         setTime(partes->tm_hour, partes->tm_min, partes->tm_sec);
13     } else {
14         setTime(h, m, s);
15     }
16 }
17
18 void Time::setTime(int h, int m, int s) {
19     setHour(h);
20     setMinute(m);
21     setSecond(s);
22 }
23
24 void Time::setHour(int h) { hour = (h >= 0 && h < 24) ? h : 0; }
25 void Time::setMinute(int m) { minute = (m >= 0 && m < 60) ? m : 0; }
26 void Time::setSecond(int s) { second = (s >= 0 && s < 60) ? s : 0; }
27
28 int Time::getHour() const { return hour; }
29 int Time::getMinute() const { return minute; }
30 int Time::getSecond() const { return second; }
31
32 void Time::printUniversal() const {
33     cout << setfill('0') << setw(2) << hour << ':'
34         << setw(2) << minute << ':'
35         << setw(2) << second;
36 }
37
38 void Time::printStandard() const {
39     int h12 = ((hour == 0 || hour == 12) ? 12 : hour % 12);

```

```

40     cout << setfill('0') << setw(2) << h12 << ':'
41         << setw(2) << minute << ':' << setw(2) << second
42         << (hour < 12 ? " AM" : " PM");
43 }

```

Programa de teste main_Time04.cpp

```

1  // Exercício 17.4 - main_Time04.cpp
2  #include <iostream>
3  #include "Time.h"
4  using namespace std;
5
6  int main() {
7      // Objeto com hora especifica
8      Time t1(false, 14, 30, 25);
9      cout << "Tempo especifico:\n";
10     cout << "    Universal: "; t1.printUniversal(); cout << "\n";
11     cout << "    Standard:  "; t1.printStandard();  cout << "\n";
12
13     // Objeto inicializado com a hora corrente do sistema
14     Time t2(true);
15     cout << "\nHora corrente do sistema:\n";
16     cout << "    Universal: "; t2.printUniversal(); cout << "\n";
17     cout << "    Standard:  "; t2.printStandard();  cout << "\n";
18
19     return 0;
20 }

```

Execução

Saída da execução

```

$ ./test_Time04
Tempo especifico:
  Universal: 14:30:25
  Standard:  02:30:25 PM

Hora corrente do sistema:
  Universal: 05:41:21
  Standard:  05:41:21 AM

```

Discussão

- Marcamos o construtor com `explicit` para impedir conversões implícitas indesejadas. Por exemplo, `Time t = true;` (que seria estranho) *não* compila com `explicit`; obriga o programador a escrever `Time t(true)`.
- A função `localtime` retorna um ponteiro para uma *estrutura estática interna*, o que significa: chamadas concorrentes ou aninhadas podem corromper o resultado. Em código de produção multithread, prefira `localtime_r` (POSIX) ou `localtime_s` (Windows).
- Note como reusamos `setTime` dentro do construtor — esse é o padrão de *delegação a setters* que começamos no Cap. 16: uma só lógica de validação em um único lugar.

Dica de Desempenho

Para programas que pedem repetidas vezes a hora atual (loops de *logging*, métricas de desempenho), considere chamar `time()` uma só vez por *frame* e reutilizar o valor. Cada chamada a `time()` é uma chamada de sistema, com custo pequeno mas não desprezível quando feita milhões de vezes.

3 Exercício 17.5 — Classe Complex

Enunciado. Crie a classe `Complex` para aritmética com números complexos $a + bi$. Use `double` privados para parte real e imaginária. Forneça construtor com defaults e funções públicas para: (a) somar; (b) subtrair; (c) imprimir no formato `(a, b)`.

Análise do problema

Um número complexo $z = a + bi$ é caracterizado por dois reais. As operações a implementar são componente a componente: $(a_1 + b_1i) + (a_2 + b_2i) = (a_1 + a_2) + (b_1 + b_2)i$ e similarmente para subtração.

Cabeçalho `Complex.h`

```

1 // Exercício 17.5 - Complex.h
2 // Classe Complex para aritmetica com numeros complexos
3 #ifndef COMPLEX_H
4 #define COMPLEX_H
5
6 class Complex {
7 public:
8     // Construtor com valores default
9     Complex(double real = 0.0, double imag = 0.0);
10
11     Complex add(const Complex &outro) const;
12     Complex subtract(const Complex &outro) const;
13
14     void print() const;    // imprime no formato (a, b)
15
16 private:
17     double parteReal;
18     double parteImaginaria;
19 };
20
21 #endif

```

Implementação `Complex.cpp`

```

1 // Exercício 17.5 - Complex.cpp
2 #include <iostream>
3 #include "Complex.h"
4 using namespace std;
5
6 Complex::Complex(double real, double imag)
7     : parteReal(real), parteImaginaria(imag) {
8 }
9
10 Complex Complex::add(const Complex &outro) const {
11     return Complex(parteReal + outro.parteReal,
12                   parteImaginaria + outro.parteImaginaria);
13 }
14
15 Complex Complex::subtract(const Complex &outro) const {
16     return Complex(parteReal - outro.parteReal,
17                   parteImaginaria - outro.parteImaginaria);
18 }
19
20 void Complex::print() const {

```

```

21     cout << "(" << parteReal << ", " << parteImaginaria << ")";
22 }

```

Programa de teste

```

1  // Exercício 17.5 - main_Complex.cpp
2  #include <iostream>
3  #include "Complex.h"
4  using namespace std;
5
6  int main() {
7      Complex c1(3.5, 7.2);
8      Complex c2(1.5, 2.0);
9      Complex c3;                // usa default: (0.0, 0.0)
10
11     cout << "c1 = "; c1.print(); cout << endl;
12     cout << "c2 = "; c2.print(); cout << endl;
13     cout << "c3 = "; c3.print(); cout << " (default)" << endl;
14
15     Complex soma = c1.add(c2);
16     cout << "\nc1 + c2 = "; soma.print(); cout << endl;
17
18     Complex diff = c1.subtract(c2);
19     cout << "c1 - c2 = "; diff.print(); cout << endl;
20
21     return 0;
22 }

```

Execução

Saída da execução

```

c1 = (3.5, 7.2)
c2 = (1.5, 2)
c3 = (0, 0) (default)

c1 + c2 = (5, 9.2)
c1 - c2 = (2, 5.2)

```

Discussão

- As funções `add` e `subtract` *não* modificam o objeto que as chama; criam e retornam um *novo* objeto `Complex`. Por isso estão marcadas `const`.
- O argumento `const Complex &outro` é a forma idiomática em C++: *const* promete que não haverá modificação do operando, e *referência* evita cópia.
- Em C++ *de produção*, deveríamos sobrecarregar os operadores `+` e `-` para escrever `c1 + c2` em vez de `c1.add(c2)`. Esse é tema do Capítulo 18 (sobrecarga de operadores). A biblioteca padrão *já* oferece a classe `std::complex<double>` pronta em `<complex>`.

Observação de Engenharia de Software

Sobre números complexos. O lembrete do enunciado de que $i = \sqrt{-1}$ é mais profundo do que parece. Antes do século XVI, equações como $x^2 + 1 = 0$ eram simplesmente declaradas “sem solução”. A invenção do número imaginário expandiu o sistema numérico, revelando

que o plano complexo é um *tipo composto natural* — exatamente como nossas classes! Daí a beleza didática de modelar `Complex` como classe: o conceito matemático e a abstração de programação coincidem.

4 Exercício 17.6 — Classe Rational

Enunciado. Crie a classe `Rational` para aritmética com frações. Use inteiros para numerador e denominador. Construtor com defaults; armazena sempre na forma reduzida. Operações: (a) somar; (b) subtrair; (c) multiplicar; (d) dividir; (e) imprimir como `a/b`; (f) imprimir como ponto flutuante.

Análise do problema

A redução da fração requer o **máximo divisor comum** (MDC). Usaremos o algoritmo de Euclides clássico: repetidamente substitui-se (a, b) por $(b, a \bmod b)$ até que $b = 0$.

As fórmulas são as elementares: $\frac{a}{b} + \frac{c}{d} = \frac{ad+bc}{bd}$, $\frac{a}{b} \cdot \frac{c}{d} = \frac{ac}{bd}$, $\frac{a}{b} \div \frac{c}{d} = \frac{ad}{bc}$. A divisão requer c (numerador do divisor) não-nulo, sob pena de divisão por zero.

Cabeçalho `Rational.h`

```

1 // Exercício 17.6 - Rational.h
2 // Classe Rational para aritmetica com fracoes, sempre em forma reduzida
3 #ifndef RATIONAL_H
4 #define RATIONAL_H
5
6 class Rational {
7 public:
8     // Construtor com valores default (numerador=0, denominador=1)
9     Rational(int numerador = 0, int denominador = 1);
10
11     Rational add(const Rational &o) const;
12     Rational subtract(const Rational &o) const;
13     Rational multiply(const Rational &o) const;
14     Rational divide(const Rational &o) const;
15
16     void printFraction() const;           // a/b
17     void printFloatingPoint() const;      // a/b como double
18
19 private:
20     int num;           // numerador
21     int den;           // denominador (sempre > 0)
22
23     void reduzir();     // simplifica num/den
24     static int mdc(int a, int b); // maior divisor comum
25 };
26
27 #endif

```

Implementação `Rational.cpp`

```

1 // Exercício 17.6 - Rational.cpp
2 #include <iostream>
3 #include <cstdlib>
4 #include "Rational.h"
5 using namespace std;
6
7 Rational::Rational(int numerador, int denominador)
8     : num(numerador), den(denominador) {
9     if (den == 0) {
10         cout << "Erro: denominador 0. Ajustado para 1." << endl;
11         den = 1;

```

```

12     }
13     // Sinal negativo sempre no numerador
14     if (den < 0) { num = -num; den = -den; }
15     reduzir();
16 }
17
18 int Rational::mdc(int a, int b) {
19     a = abs(a); b = abs(b);
20     while (b != 0) { int t = b; b = a % b; a = t; }
21     return a == 0 ? 1 : a;
22 }
23
24 void Rational::reduzir() {
25     int d = mdc(num, den);
26     num /= d;
27     den /= d;
28 }
29
30 Rational Rational::add(const Rational &o) const {
31     return Rational(num * o.den + o.num * den, den * o.den);
32 }
33
34 Rational Rational::subtract(const Rational &o) const {
35     return Rational(num * o.den - o.num * den, den * o.den);
36 }
37
38 Rational Rational::multiply(const Rational &o) const {
39     return Rational(num * o.num, den * o.den);
40 }
41
42 Rational Rational::divide(const Rational &o) const {
43     return Rational(num * o.den, den * o.num);
44 }
45
46 void Rational::printFraction() const {
47     cout << num << "/" << den;
48 }
49
50 void Rational::printFloatingPoint() const {
51     cout << static_cast<double>(num) / den;
52 }

```

Programa de teste

```

1  // Exercício 17.6 - main_Rational.cpp
2  #include <iostream>
3  #include "Rational.h"
4  using namespace std;
5
6  void mostrar(const char *label, const Rational &r) {
7      cout << label << " = ";
8      r.printFraction();
9      cout << " (";
10     r.printFloatingPoint();
11     cout << ")" << endl;
12 }
13
14 int main() {
15     cout << "=== Teste da classe Rational ===" << endl;
16
17     // Construtor com reducao automatica
18     Rational r1(2, 4);           // deve virar 1/2

```

```

19 Rational r2(3, 9);           // deve virar 1/3
20 Rational r3;                 // default: 0/1
21
22 cout << "\nConstrutores (verifica reducao automatica):" << endl;
23 mostrar("2/4 reduzido", r1);
24 mostrar("3/9 reduzido", r2);
25 mostrar("default      ", r3);
26
27 cout << "\nOperacoes aritmeticas:" << endl;
28 mostrar("r1 + r2", r1.add(r2));
29 mostrar("r1 - r2", r1.subtract(r2));
30 mostrar("r1 * r2", r1.multiply(r2));
31 mostrar("r1 / r2", r1.divide(r2));
32
33 // Casos com sinais negativos
34 cout << "\nCom sinais negativos:" << endl;
35 Rational r4(-6, 8);           // -3/4
36 Rational r5(6, -8);           // -3/4 (sinal no num)
37 mostrar("-6/8 ", r4);
38 mostrar("6/-8 ", r5);
39
40 return 0;
41 }

```

Execução

Saída da execução

```

=== Teste da classe Rational ===

Construtores (verifica reducao automatica):
2/4 reduzido = 1/2   (0.5)
3/9 reduzido = 1/3   (0.333333)
default      = 0/1   (0)

Operacoes aritmeticas:
r1 + r2 = 5/6   (0.833333)
r1 - r2 = 1/6   (0.166667)
r1 * r2 = 1/6   (0.166667)
r1 / r2 = 3/2   (1.5)

Com sinais negativos:
-6/8  = -3/4   (-0.75)
6/-8  = -3/4   (-0.75)

```

Discussão

Conferência matemática: $\frac{1}{2} + \frac{1}{3} = \frac{3+2}{6} = \frac{5}{6}$; $\frac{1}{2} - \frac{1}{3} = \frac{1}{6}$; $\frac{1}{2} \cdot \frac{1}{3} = \frac{1}{6}$; $\frac{1}{2} \div \frac{1}{3} = \frac{1}{2} \cdot \frac{3}{1} = \frac{3}{2}$.

- Note o uso de `static` em `mdc`: é uma função auxiliar que não precisa de `this` (não opera sobre nenhum objeto específico). Marcá-la como `static` é a forma C++ de torná-la uma função auxiliar pertencente à classe.
- Estratégia de sinais: o sinal negativo, se houver, sempre fica no numerador. Isso simplifica comparações e impressão. Observe a aritmética: $\frac{-6}{8}$ e $\frac{6}{-8}$ são ambos $-\frac{3}{4}$.
- A redução é aplicada *no construtor*, não apenas após as operações: `add`, `subtract` etc. constroem um novo `Rational`, e o construtor reduz. Garantia: *nenhum* objeto `Rational` jamais sai do construtor em forma não reduzida.

Erro Comum de Programação

Em C++ *de produção*, multiplicar inteiros pode causar *overflow* sem aviso. `add` faz `num*o.den + o.num*den`: se `num`, `den` forem da ordem de 10^5 , o produto pode passar de 10^{10} , que ultrapassa o `int` de 32 bits. Código robusto usaria `long long` ou inteiros de precisão arbitrária (ver Ex. 17.14).

5 Exercício 17.7 — Time::tick

Enunciado. Modifique a classe `Time` (figs. 17.8/17.9) para incluir uma função-membro `tick` que incremente a hora armazenada em 1 segundo. O objeto deve permanecer sempre coerente. Teste em loop, mostrando: (a) transição para o próximo minuto; (b) transição para a próxima hora; (c) transição de 23:59:59 para 00:00:00.

Análise do problema

A lógica do `tick` é uma cascata de propagações: incrementa segundo; se chegou a 60, zera e incrementa minuto; se chegou a 60, zera e incrementa hora; se chegou a 24, zera (volta para o próximo dia). Em `tick`, não tratamos a transição de dia para dia ainda — isso virará no Ex. 17.9 quando combinarmos com `Date`.

Cabeçalho TimeTick.h

```

1 // Exercício 17.7 - TimeTick.h
2 // Classe Time com funcao-membro tick que incrementa em 1 segundo
3 #ifndef TIMETICK_H
4 #define TIMETICK_H
5
6 class TimeTick {
7 public:
8     TimeTick(int h = 0, int m = 0, int s = 0);
9
10    void setTime(int h, int m, int s);
11    int getHour() const;
12    int getMinute() const;
13    int getSecond() const;
14
15    // Incrementa o tempo em 1 segundo, propagando para min/hora/dia
16    void tick();
17
18    void printUniversal() const;
19    void printStandard() const;
20
21 private:
22     int hour;
23     int minute;
24     int second;
25 };
26
27 #endif

```

Implementação TimeTick.cpp

```

1 // Exercício 17.7 - TimeTick.cpp
2 #include <iostream>
3 #include <iomanip>
4 #include "TimeTick.h"
5 using namespace std;
6
7 TimeTick::TimeTick(int h, int m, int s) {
8     setTime(h, m, s);
9 }
10
11 void TimeTick::setTime(int h, int m, int s) {

```

```

12     hour    = (h >= 0 && h < 24) ? h : 0;
13     minute  = (m >= 0 && m < 60) ? m : 0;
14     second  = (s >= 0 && s < 60) ? s : 0;
15 }
16
17 int TimeTick::getHour()    const { return hour;    }
18 int TimeTick::getMinute() const { return minute; }
19 int TimeTick::getSecond() const { return second; }
20
21 void TimeTick::tick() {
22     ++second;
23     if (second == 60) {
24         second = 0;
25         ++minute;
26         if (minute == 60) {
27             minute = 0;
28             ++hour;
29             if (hour == 24) {
30                 hour = 0;        // volta para 00:00:00
31             }
32         }
33     }
34 }
35
36 void TimeTick::printUniversal() const {
37     cout << setfill('0')
38         << setw(2) << hour    << ':'
39         << setw(2) << minute << ':'
40         << setw(2) << second
41         << setfill(' ');
42 }
43
44 void TimeTick::printStandard() const {
45     int h12 = ((hour == 0 || hour == 12) ? 12 : hour % 12);
46     cout << setfill('0')
47         << setw(2) << h12    << ':'
48         << setw(2) << minute << ':'
49         << setw(2) << second
50         << (hour < 12 ? " AM" : " PM")
51         << setfill(' ');
52 }

```

Programa de teste

```

1  // Exercício 17.7 - main_TimeTick.cpp
2  // Testa a funcao tick em situacoes criticas
3  #include <iostream>
4  #include "TimeTick.h"
5  using namespace std;
6
7  void testarTransicao(const char *etiqueta, int h, int m, int s, int qtos) {
8      cout << "\n=== " << etiqueta << " ===" << endl;
9      TimeTick t(h, m, s);
10     cout << "Inicio: "; t.printUniversal(); cout << endl;
11     for (int i = 0; i < qtos; ++i) {
12         t.tick();
13         cout << " Tick #" << (i+1) << ": "; t.printUniversal();
14         cout << " ("; t.printStandard(); cout << ")" << endl;
15     }
16 }
17
18 int main() {

```

```

19 // (a) Incrementar para o próximo minuto
20 testarTransicao("Transicao de minuto (11:30:58 -> 11:31:00)",
21               11, 30, 58, 3);
22
23 // (b) Incrementar para a próxima hora
24 testarTransicao("Transicao de hora (08:59:58 -> 09:00:00)",
25               8, 59, 58, 3);
26
27 // (c) Incrementar de 23:59:59 para 00:00:00
28 testarTransicao("Transicao de dia (23:59:58 -> 00:00:00)",
29               23, 59, 58, 3);
30
31 return 0;
32 }

```

Execução

Saída da execução

```

=== Transicao de minuto (11:30:58 -> 11:31:00) ===
Inicio: 11:30:58
Tick #1: 11:30:59 (11:30:59 AM)
Tick #2: 11:31:00 (11:31:00 AM)
Tick #3: 11:31:01 (11:31:01 AM)

=== Transicao de hora (08:59:58 -> 09:00:00) ===
Inicio: 08:59:58
Tick #1: 08:59:59 (08:59:59 AM)
Tick #2: 09:00:00 (09:00:00 AM)
Tick #3: 09:00:01 (09:00:01 AM)

=== Transicao de dia (23:59:58 -> 00:00:00) ===
Inicio: 23:59:58
Tick #1: 23:59:59 (11:59:59 PM)
Tick #2: 00:00:00 (12:00:00 AM)
Tick #3: 00:00:01 (12:00:01 AM)

```

Discussão

A cascata de `if` aninhados é *não casual*: cada nível só pode disparar se o nível anterior também disparou. Para os três casos de transição acima, observe:

- No primeiro tick de 11:30:58, `second` vira 59 — nada propaga.
- No segundo tick, `second` vira 60 → zero, `minute` sobe a 31 — só um nível propagou.
- Em 23:59:59, todos os três níveis propagam de uma vez: `second` 60→0, `minute` 60→0, `hour` 24→0.

Observação de Engenharia de Software

A invariante de classe que estamos preservando é: `0 <= hour < 24 && 0 <= minute < 60 && 0 <= second < 60`. Diz-se que o objeto está em **estado coerente** quando essa invariante é válida. Toda função-membro pública que possa modificar o estado tem a obrigação de recebê-lo coerente e devolvê-lo coerente. Esse é um dos princípios mais profundos da orientação a objetos: classes encapsulam invariantes.

6 Exercício 17.8 — Date::nextDay

Enunciado. Modifique a classe `Date` (figs. 17.17/17.18) para fazer verificação de erro nos inicializadores de `month`, `day` e `year`. Forneça uma função-membro `nextDay` que incrementa o dia em um. Teste com transições: (a) para o próximo mês; (b) para o próximo ano.

Análise do problema

Diferentemente do `tick` do tempo, o número de dias por mês *varia*: 31, 30, 28 (29 em ano bissexto). Precisamos então de uma função auxiliar `diasDoMes` que retorne o limite correto para o mês atual e detecte anos bissextos pela regra clássica: divisível por 4, exceto se divisível por 100 e não por 400 (2000 é bissexto, 1900 não).

Cabeçalho DateNext.h

```

1 // Exercício 17.8 - DateNext.h
2 // Classe Date com verificacao de erro e funcao nextDay
3 #ifndef DATENEXT_H
4 #define DATENEXT_H
5
6 class DateNext {
7 public:
8     DateNext(int m = 1, int d = 1, int y = 2000);
9
10    void setMonth(int m);
11    void setDay(int d);
12    void setYear(int y);
13    int getMonth() const;
14    int getDay() const;
15    int getYear() const;
16
17    // Avanca a data em um dia, propagando para mes/ano
18    void nextDay();
19    void print() const;      // formato mm/dd/yyyy
20
21 private:
22     int month;
23     int day;
24     int year;
25
26     // Quantos dias tem este mes (considerando ano bissexto)
27     int diasDoMes() const;
28     static bool ehBissexto(int y);
29 };
30
31 #endif

```

Implementação DateNext.cpp

```

1 // Exercício 17.8 - DateNext.cpp
2 #include <iostream>
3 #include "DateNext.h"
4 using namespace std;
5
6 DateNext::DateNext(int m, int d, int y) {
7     setYear(y);
8     setMonth(m);
9     setDay(d);

```



```

10 }
11
12 bool DateNext::ehBissexto(int y) {
13     return (y % 4 == 0 && y % 100 != 0) || (y % 400 == 0);
14 }
15
16 int DateNext::diasDoMes() const {
17     static const int dias[] = {31,28,31,30,31,30,31,31,30,31,30,31};
18     if (month == 2 && ehBissexto(year)) return 29;
19     return dias[month - 1];
20 }
21
22 void DateNext::setMonth(int m) {
23     if (m >= 1 && m <= 12) month = m;
24     else {
25         cout << "Mes invalido (" << m << "). Definido como 1." << endl;
26         month = 1;
27     }
28 }
29
30 void DateNext::setDay(int d) {
31     int max = diasDoMes();
32     if (d >= 1 && d <= max) day = d;
33     else {
34         cout << "Dia invalido (" << d << ") para mes " << month
35             << "/" << year << ". Definido como 1." << endl;
36         day = 1;
37     }
38 }
39
40 void DateNext::setYear(int y) {
41     if (y >= 1) year = y;
42     else {
43         cout << "Ano invalido (" << y << "). Definido como 2000." << endl;
44         year = 2000;
45     }
46 }
47
48 int DateNext::getMonth() const { return month; }
49 int DateNext::getDay() const { return day; }
50 int DateNext::getYear() const { return year; }
51
52 void DateNext::nextDay() {
53     ++day;
54     if (day > diasDoMes()) {
55         day = 1;
56         ++month;
57         if (month > 12) {
58             month = 1;
59             ++year;
60         }
61     }
62 }
63
64 void DateNext::print() const {
65     cout << month << "/" << day << "/" << year;
66 }

```

Programa de teste

```

1 // Exercício 17.8 - main_DateNext.cpp
2 #include <iostream>

```

```

3  #include "DateNext.h"
4  using namespace std;
5
6  void testar(const char *etiqueta, int m, int d, int y, int qtos) {
7      cout << "\n=== " << etiqueta << " ===" << endl;
8      DateNext data(m, d, y);
9      cout << "Inicio: "; data.print(); cout << endl;
10     for (int i = 0; i < qtos; ++i) {
11         data.nextDay();
12         cout << " nextDay #" << (i+1) << ": "; data.print(); cout << endl;
13     }
14 }
15
16 int main() {
17     // (a) Transicao de mes (fim de marco -> abril)
18     testar("Transicao de mes (30/3 -> 1/4)", 3, 30, 2026, 3);
19
20     // (b) Transicao de ano (fim de dezembro -> janeiro)
21     testar("Transicao de ano (30/12 -> 1/1/2027)", 12, 30, 2026, 3);
22
23     // Fevereiro em ano nao bissexto (28 dias)
24     testar("Fevereiro nao bissexto (27/2/2025 -> 1/3)", 2, 27, 2025, 3);
25
26     // Fevereiro em ano bissexto (29 dias)
27     testar("Fevereiro bissexto (28/2/2024 -> 29/2 -> 1/3)", 2, 28, 2024, 2);
28
29     // Testes de validacao no construtor
30     cout << "\n=== Testes de validacao ===" << endl;
31     DateNext invalida(13, 32, 2025); // mes invalido, dia tambem
32     cout << "Apos validacao: "; invalida.print(); cout << endl;
33
34     return 0;
35 }

```

Execução

Saída da execução

```

=== Transicao de mes (30/3 -> 1/4) ===
Inicio: 3/30/2026
 nextDay #1: 3/31/2026
 nextDay #2: 4/1/2026
 nextDay #3: 4/2/2026

=== Transicao de ano (30/12 -> 1/1/2027) ===
Inicio: 12/30/2026
 nextDay #1: 12/31/2026
 nextDay #2: 1/1/2027
 nextDay #3: 1/2/2027

=== Fevereiro nao bissexto (27/2/2025 -> 1/3) ===
Inicio: 2/27/2025
 nextDay #1: 2/28/2025
 nextDay #2: 3/1/2025
 nextDay #3: 3/2/2025

=== Fevereiro bissexto (28/2/2024 -> 29/2 -> 1/3) ===
Inicio: 2/28/2024
 nextDay #1: 2/29/2024
 nextDay #2: 3/1/2024

```

```
=== Testes de validacao ===  
Mes invalido (13). Definido como 1.  
Dia invalido (32) para mes 1/2025. Definido como 1.  
Apos validacao: 1/1/2025
```

Discussão

- A ordem de validação no construtor é crítica: `setYear` antes de `setDay`, e `setMonth` antes de `setDay`. Porque o limite máximo do dia depende do mês e do ano. Se valiçássemos o dia primeiro, não saberíamos se 29/2 é válido (depende do ano).
- Bissexto pela regra do calendário gregoriano: 2024 é bissexto (divisível por 4, não por 100), 2025 não é. A saída confirma: `nextDay` sobre 28/2/2024 entrega 29/2/2024, mas sobre 28/2/2025 já salta para 1/3/2025.
- A array estática `static const int dias[]` dentro de `diasDoMes`: o modificador `static` aqui significa “inicializada uma vez na primeira chamada e mantida entre chamadas”.

7 Exercício 17.9 — DateAndTime: combinação de Time e Date

Enunciado. Combine a classe `Time` (modificada no Ex. 17.7) e a classe `Date` (modificada no Ex. 17.8) em uma só classe `DateAndTime`. Modifique `tick` para chamar `nextDay` quando a hora avançar para o dia seguinte. Modifique `printStandard` e `printUniversal` para mostrar data e hora. Teste especificamente a passagem de hora para o dia seguinte.

Análise do problema

O enunciado já antecipa: “No Cap. 20 discutiremos herança, que permite essa combinação rapidamente”. Como ainda não temos herança, copiamos os dados e funções das duas classes para uma terceira. É uma abordagem manualmente repetitiva mas pedagogicamente útil: deixa explícita a duplicação que a herança vai *eliminar* no futuro.

A nova lógica chave: ao `tick`, quando `hour` chega a 24, em vez de simplesmente zerar, chamamos `nextDay()`, que avança a data.

Cabeçalho `DateAndTime.h`

```

1  // Exercício 17.9 - DateAndTime.h
2  // Combina Date e Time em uma unica classe; tick chama nextDay quando passa de
   23:59:59
3  #ifndef DATEANDTIME_H
4  #define DATEANDTIME_H
5
6  class DateAndTime {
7  public:
8      DateAndTime(int mes = 1, int dia = 1, int ano = 2000,
9                  int h = 0, int m = 0, int s = 0);
10
11     // Time
12     void setTime(int h, int m, int s);
13     int  getHour() const;
14     int  getMinute() const;
15     int  getSecond() const;
16
17     // Date
18     void setDate(int mes, int dia, int ano);
19     int  getMonth() const;
20     int  getDay() const;
21     int  getYear() const;
22
23     // Tick e nextDay
24     void tick();           // incrementa 1 segundo; chama nextDay no fim do dia
25     void nextDay();       // avanca 1 dia, propagando mes/ano
26
27     void printStandard() const;
28     void printUniversal() const;
29
30 private:
31     int hour, minute, second;
32     int month, day, year;
33
34     int  diasDoMes() const;
35     static bool ehBissexto(int y);
36 };
37
38 #endif

```

Implementação DateAndTime.cpp

```

1  // Exercício 17.9 - DateAndTime.cpp
2  #include <iostream>
3  #include <iomanip>
4  #include "DateAndTime.h"
5  using namespace std;
6
7  DateAndTime::DateAndTime(int mes, int dia, int ano, int h, int m, int s) {
8      setDate(mes, dia, ano);
9      setTime(h, m, s);
10 }
11
12 bool DateAndTime::ehBissexto(int y) {
13     return (y % 4 == 0 && y % 100 != 0) || (y % 400 == 0);
14 }
15
16 int DateAndTime::diasDoMes() const {
17     static const int dias[] = {31,28,31,30,31,30,31,31,30,31,30,31};
18     if (month == 2 && ehBissexto(year)) return 29;
19     return dias[month - 1];
20 }
21
22 void DateAndTime::setTime(int h, int m, int s) {
23     hour   = (h >= 0 && h < 24) ? h : 0;
24     minute = (m >= 0 && m < 60) ? m : 0;
25     second = (s >= 0 && s < 60) ? s : 0;
26 }
27
28 int DateAndTime::getHour()   const { return hour; }
29 int DateAndTime::getMinute() const { return minute; }
30 int DateAndTime::getSecond() const { return second; }
31
32 void DateAndTime::setDate(int mes, int dia, int ano) {
33     year = (ano >= 1) ? ano : 2000;
34     month = (mes >= 1 && mes <= 12) ? mes : 1;
35     day   = (dia >= 1 && dia <= diasDoMes()) ? dia : 1;
36 }
37
38 int DateAndTime::getMonth() const { return month; }
39 int DateAndTime::getDay()   const { return day; }
40 int DateAndTime::getYear()  const { return year; }
41
42 void DateAndTime::nextDay() {
43     ++day;
44     if (day > diasDoMes()) {
45         day = 1;
46         ++month;
47         if (month > 12) {
48             month = 1;
49             ++year;
50         }
51     }
52 }
53
54 void DateAndTime::tick() {
55     ++second;
56     if (second == 60) {
57         second = 0;
58         ++minute;
59         if (minute == 60) {
60             minute = 0;
61             ++hour;
62             if (hour == 24) {

```

```

63         hour = 0;
64         nextDay();    // delega para nextDay
65     }
66 }
67 }
68 }
69
70 void DateAndTime::printUniversal() const {
71     cout << month << "/" << day << "/" << year << " "
72         << setfill('0')
73         << setw(2) << hour << ':'
74         << setw(2) << minute << ':'
75         << setw(2) << second
76         << setfill(' ');
77 }
78
79 void DateAndTime::printStandard() const {
80     int h12 = ((hour == 0 || hour == 12) ? 12 : hour % 12);
81     cout << month << "/" << day << "/" << year << " "
82         << setfill('0')
83         << setw(2) << h12 << ':'
84         << setw(2) << minute << ':'
85         << setw(2) << second
86         << (hour < 12 ? " AM" : " PM")
87         << setfill(' ');
88 }

```

Programa de teste

```

1  // Exercicio 17.9 - main_DateAndTime.cpp
2  #include <iostream>
3  #include "DateAndTime.h"
4  using namespace std;
5
6  int main() {
7      cout << "=== Teste 1: transicao normal de hora ===" << endl;
8      DateAndTime dt1(5, 15, 2026, 9, 59, 58);
9      cout << "Inicio: "; dt1.printUniversal(); cout << endl;
10     for (int i = 0; i < 3; ++i) {
11         dt1.tick();
12         cout << " Tick #" << (i+1) << ": "; dt1.printUniversal();
13         cout << endl;
14     }
15
16     cout << "\n=== Teste 2: transicao 23:59:59 -> dia seguinte ===" << endl;
17     DateAndTime dt2(5, 15, 2026, 23, 59, 58);
18     cout << "Inicio: "; dt2.printUniversal(); cout << endl;
19     for (int i = 0; i < 4; ++i) {
20         dt2.tick();
21         cout << " Tick #" << (i+1) << ": "; dt2.printUniversal();
22         cout << endl;
23     }
24
25     cout << "\n=== Teste 3: transicao 31/12 23:59:58 -> 1/1/2027 ===" << endl;
26     DateAndTime dt3(12, 31, 2026, 23, 59, 58);
27     cout << "Inicio: "; dt3.printUniversal(); cout << endl;
28     for (int i = 0; i < 4; ++i) {
29         dt3.tick();
30         cout << " Tick #" << (i+1) << ": "; dt3.printUniversal();
31         cout << " ("; dt3.printStandard(); cout << ")" << endl;
32     }
33 }

```

```
34     return 0;
35 }
```

Execução

Saída da execução

```
=== Teste 1: transicao normal de hora ===
Inicio: 5/15/2026 09:59:58
Tick #1: 5/15/2026 09:59:59
Tick #2: 5/15/2026 10:00:00
Tick #3: 5/15/2026 10:00:01

=== Teste 2: transicao 23:59:59 -> dia seguinte ===
Inicio: 5/15/2026 23:59:58
Tick #1: 5/15/2026 23:59:59
Tick #2: 5/16/2026 00:00:00
Tick #3: 5/16/2026 00:00:01
Tick #4: 5/16/2026 00:00:02

=== Teste 3: transicao 31/12 23:59:58 -> 1/1/2027 ===
Inicio: 12/31/2026 23:59:58
Tick #1: 12/31/2026 23:59:59 (12/31/2026 11:59:59 PM)
Tick #2: 1/1/2027 00:00:00 (1/1/2027 12:00:00 AM)
Tick #3: 1/1/2027 00:00:01 (1/1/2027 12:00:01 AM)
Tick #4: 1/1/2027 00:00:02 (1/1/2027 12:00:02 AM)
```

Discussão

Observe o teste mais crítico: *transição de ano* (31/12/2026 23:59:58). Em um único tick, todas as casquinhas explodem em cascata: `second` 60→0, `minute` 60→0, `hour` 24→0, `nextDay` dispara, `day` excede 31 (dezembro), `month` 13→1, `year` avança. Todas as transições acertaram — a saída é 1/1/2027 00:00:00 (12:00:00 AM).

Observação de Engenharia de Software

Por que copiamos em vez de herdar? Herança, vista no Capítulo 20, permite criar `DateAndTime` como “filho” de `Date` e `Time` (herança múltipla), reaproveitando todos os métodos. O fato de o enunciado escolher a abordagem manual aqui *deliberadamente* faz o leitor *sentir* a duplicação — e essa sensação motiva a aceitação posterior da herança como solução. Toda boa abstração em programação surge da rejeição de uma duplicação previamente experimentada.

8 Exercício 17.10 — Time com *setters* que retornam erro

Enunciado. Modifique as funções *set* na classe `Time` para retornar valores de erro apropriados se for feita uma tentativa de definir um dado-membro para valor inválido. Escreva um programa de teste que mostre mensagens de erro quando as funções *set* retornam valores de erro.

Análise do problema

Até agora, nossas validações silenciosamente ajustavam o valor inválido (a 0) e, no caso de classes como `Date`, imprimiam uma mensagem. Aqui o enunciado pede um padrão diferente: cada *setter* deve *retornar* um código de erro, permitindo ao *chamador* reagir como desejar. É uma forma de tornar o código cliente *informado* dos erros, em vez de aceitá-los silenciosamente.

Adotaremos uma *enumeração* com bits independentes: `ERR_HOUR = 1`, `ERR_MINUTE = 2`, `ERR_SECOND = 4`. A combinação via OR bit-a-bit (`|`) permite que `setTime` reporte simultaneamente *todos* os erros que ocorreram.

Cabeçalho `TimeWithErrors.h`

```

1  // Exercício 17.10 - TimeWithErrors.h
2  // Classe Time com setters que retornam código de erro
3  #ifndef TIMEWITHEERRORS_H
4  #define TIMEWITHEERRORS_H
5
6  class TimeWithErrors {
7  public:
8      // Codigos de erro retornados pelos setters
9      enum ErrorCode {
10         SUCCESS = 0,
11         ERR_HOUR = 1,
12         ERR_MINUTE = 2,
13         ERR_SECOND = 4
14     };
15
16     TimeWithErrors(int h = 0, int m = 0, int s = 0);
17
18     // Cada setter retorna 0 se OK, ou um código de erro != 0
19     int setTime(int h, int m, int s);
20     int setHour(int h);
21     int setMinute(int m);
22     int setSecond(int s);
23
24     int getHour() const;
25     int getMinute() const;
26     int getSecond() const;
27
28     void printUniversal() const;
29     void printStandard() const;
30
31 private:
32     int hour;
33     int minute;
34     int second;
35 };
36
37 #endif

```


Implementação TimeWithErrors.cpp

```

1  // Exercício 17.10 - TimeWithErrors.cpp
2  #include <iostream>
3  #include <iomanip>
4  #include "TimeWithErrors.h"
5  using namespace std;
6
7  TimeWithErrors::TimeWithErrors(int h, int m, int s) {
8      setTime(h, m, s);
9  }
10
11 int TimeWithErrors::setHour(int h) {
12     if (h >= 0 && h < 24) { hour = h; return SUCCESS; }
13     hour = 0;
14     return ERR_HOUR;
15 }
16
17 int TimeWithErrors::setMinute(int m) {
18     if (m >= 0 && m < 60) { minute = m; return SUCCESS; }
19     minute = 0;
20     return ERR_MINUTE;
21 }
22
23 int TimeWithErrors::setSecond(int s) {
24     if (s >= 0 && s < 60) { second = s; return SUCCESS; }
25     second = 0;
26     return ERR_SECOND;
27 }
28
29 int TimeWithErrors::setTime(int h, int m, int s) {
30     // Combina os códigos de erro com OR bit-a-bit
31     return setHour(h) | setMinute(m) | setSecond(s);
32 }
33
34 int TimeWithErrors::getHour() const { return hour; }
35 int TimeWithErrors::getMinute() const { return minute; }
36 int TimeWithErrors::getSecond() const { return second; }
37
38 void TimeWithErrors::printUniversal() const {
39     cout << setfill('0')
40          << setw(2) << hour << ':'
41          << setw(2) << minute << ':'
42          << setw(2) << second
43          << setfill(' ');
44 }
45
46 void TimeWithErrors::printStandard() const {
47     int h12 = ((hour == 0 || hour == 12) ? 12 : hour % 12);
48     cout << setfill('0')
49          << setw(2) << h12 << ':'
50          << setw(2) << minute << ':'
51          << setw(2) << second
52          << (hour < 12 ? " AM" : " PM")
53          << setfill(' ');
54 }

```

Programa de teste

```

1  // Exercício 17.10 - main_TimeWithErrors.cpp
2  #include <iostream>
3  #include "TimeWithErrors.h"

```

```

4 using namespace std;
5
6 // Reporta texto humano para cada código de erro
7 void reportar(int code) {
8     if (code == TimeWithErrors::SUCCESS) {
9         cout << " OK: valor aceito." << endl;
10        return;
11    }
12    cout << " ERRO (codigo " << code << "):";
13    if (code & TimeWithErrors::ERR_HOUR) cout << " hora invalida;";
14    if (code & TimeWithErrors::ERR_MINUTE) cout << " minuto invalido;";
15    if (code & TimeWithErrors::ERR_SECOND) cout << " segundo invalido;";
16    cout << endl;
17 }
18
19 int main() {
20     TimeWithErrors t;
21
22     cout << "=== Tentativas validas e invalidas ===" << endl;
23
24     cout << "\nt.setTime(10, 30, 45):" << endl;
25     reportar(t.setTime(10, 30, 45));
26     cout << " Estado: "; t.printUniversal(); cout << endl;
27
28     cout << "\nt.setTime(25, 30, 45) [hora invalida]:" << endl;
29     reportar(t.setTime(25, 30, 45));
30     cout << " Estado: "; t.printUniversal(); cout << endl;
31
32     cout << "\nt.setTime(10, 75, 45) [minuto invalido]:" << endl;
33     reportar(t.setTime(10, 75, 45));
34     cout << " Estado: "; t.printUniversal(); cout << endl;
35
36     cout << "\nt.setTime(10, 30, 99) [segundo invalido]:" << endl;
37     reportar(t.setTime(10, 30, 99));
38     cout << " Estado: "; t.printUniversal(); cout << endl;
39
40     cout << "\nt.setTime(25, 75, 99) [tudo invalido]:" << endl;
41     reportar(t.setTime(25, 75, 99));
42     cout << " Estado: "; t.printUniversal(); cout << endl;
43
44     cout << "\nt.setHour(-5) [hora negativa]:" << endl;
45     reportar(t.setHour(-5));
46     cout << " Estado: "; t.printUniversal(); cout << endl;
47
48     return 0;
49 }

```

Execução (saída resumida)

Saída da execução

```

t.setTime(10, 30, 45):
OK: valor aceito.
Estado: 10:30:45

t.setTime(25, 30, 45) [hora invalida]:
ERRO (codigo 1): hora invalida;
Estado: 00:30:45

t.setTime(10, 75, 45) [minuto invalido]:
ERRO (codigo 2): minuto invalido;

```

```
Estado: 10:00:45

t.setTime(10, 30, 99) [segundo invalido]:
ERRO (codigo 4): segundo invalido;
Estado: 10:30:00

t.setTime(25, 75, 99) [tudo invalido]:
ERRO (codigo 7): hora invalida; minuto invalido; segundo invalido;
Estado: 00:00:00
```

Discussão

- Os códigos são potências de 2 (1, 2, 4) precisamente para que possam ser combinados sem perda: o código final 7 = 1+2+4 mostra que *os três* setters falharam. Esse padrão é chamado de *bit flags*.
- O `enum` aninhado `ErrorCode` é declarado *público* dentro da classe, então o código cliente faz `TimeWithErrors::ERR_HOUR`. É uma forma de *namespacing* natural: o nome do erro fica vinculado à classe que o gera.
- A combinação `setHour(h) | setMinute(m) | setSecond(s)` computa todos os três *antes* de aplicar o OR — o que é o que queremos (queremos chamar os três independentemente do resultado do primeiro). Se usássemos `||` (OR lógico curto-circuito) em vez de `|` (OR bit-a-bit), a avaliação pararia após o primeiro código não-zero.

Observação de Engenharia de Software

Em C++ moderno (C++11+), exceções (`throw/try/catch`) substituem em muitos cenários o padrão de código-de-erro. Mas *nem sempre* exceções são apropriadas: em código embarcado ou em *hot paths*, códigos de erro são mais previsíveis em desempenho. Ainda mais moderno: `std::optional<T>`, `std::expected<T,E>` (C++23). Tratamento de exceções é tema do Capítulo 19.

9 Exercício 17.11 — Classe Retangulo

Enunciado. Crie uma classe `Retangulo` com atributos `comprimento` e `largura`, defaults = 1. Forneça funções-membro para perímetro e área, e *set/get* para os atributos. As funções *set* devem verificar que comprimento e largura são doubles em (0,0, 20,0).

Cabeçalho `Retangulo.h`

```

1  // Exercício 17.11 - Retangulo.h
2  // Retangulo simples com comprimento e largura, defaults = 1
3  #ifndef RETANGULO_H
4  #define RETANGULO_H
5
6  class Retangulo {
7  public:
8      Retangulo(double comp = 1.0, double larg = 1.0);
9
10     // Setters validam: > 0.0 e < 20.0
11     void setComprimento(double c);
12     void setLargura(double l);
13     double getComprimento() const;
14     double getLargura() const;
15
16     double perimetro() const;
17     double area() const;
18
19 private:
20     double comprimento;
21     double largura;
22 };
23
24 #endif

```

Implementação

```

1  // Exercício 17.11 - Retangulo.cpp
2  #include <iostream>
3  #include "Retangulo.h"
4  using namespace std;
5
6  Retangulo::Retangulo(double comp, double larg) {
7      setComprimento(comp);
8      setLargura(larg);
9  }
10
11 void Retangulo::setComprimento(double c) {
12     if (c > 0.0 && c < 20.0) {
13         comprimento = c;
14     } else {
15         cout << "Comprimento invalido (" << c
16              << "). Definido como 1.0." << endl;
17         comprimento = 1.0;
18     }
19 }
20
21 void Retangulo::setLargura(double l) {
22     if (l > 0.0 && l < 20.0) {
23         largura = l;
24     } else {

```

```

25     cout << "Largura invalida (" << l
26         << "). Definida como 1.0." << endl;
27     largura = 1.0;
28 }
29 }
30
31 double Retangulo::getComprimento() const { return comprimento; }
32 double Retangulo::getLargura()      const { return largura;      }
33
34 double Retangulo::perimetro() const {
35     return 2.0 * (comprimento + largura);
36 }
37
38 double Retangulo::area() const {
39     return comprimento * largura;
40 }

```

Programa de teste

```

1  // Exercício 17.11 - main_Retangulo.cpp
2  #include <iostream>
3  #include <iomanip>
4  #include "Retangulo.h"
5  using namespace std;
6
7  void exibir(const char *etiqueta, const Retangulo &r) {
8      cout << etiqueta
9          << "   Comp=" << r.getComprimento()
10         << "   Larg=" << r.getLargura()
11         << "   Perimetro=" << r.perimetro()
12         << "   Area=" << r.area() << endl;
13 }
14
15 int main() {
16     cout << fixed << setprecision(2);
17
18     cout << "=== Construtores ===" << endl;
19     Retangulo r0;                // default 1x1
20     exibir("Default:   ", r0);
21
22     Retangulo r1(5.0, 3.0);      // valido
23     exibir("5x3:      ", r1);
24
25     cout << "\n=== Validacao ===" << endl;
26     Retangulo r2(25.0, 5.0);    // comprimento invalido
27     exibir("(25,5):   ", r2);
28
29     Retangulo r3(-1.0, -2.0);   // ambos invalidos
30     exibir("(-1,-2):  ", r3);
31
32     cout << "\n=== Setter direto ===" << endl;
33     r1.setComprimento(100.0);   // invalido
34     r1.setLargura(7.5);        // valido
35     exibir("Apos sets: ", r1);
36
37     return 0;
38 }

```

Execução

Saída da execução

```
=== Construtores ===
Default:      Comp=1.00  Larg=1.00  Perimetro=4.00  Area=1.00
5x3:          Comp=5.00  Larg=3.00  Perimetro=16.00 Area=15.00

=== Validacao ===
Comprimento invalido (25.00). Definido como 1.0.
(25,5):       Comp=1.00  Larg=5.00  Perimetro=12.00 Area=5.00
Comprimento invalido (-1.00). Definido como 1.0.
Largura invalida (-2.00). Definida como 1.0.
(-1,-2):      Comp=1.00  Larg=1.00  Perimetro=4.00  Area=1.00

=== Setter direto ===
Comprimento invalido (100.00). Definido como 1.0.
Apos sets:    Comp=1.00  Larg=7.50  Perimetro=17.00 Area=7.50
```

Discussão

Conferindo: 5×3 dá perímetro $2(5 + 3) = 16$ e área 15.

- A condição de validade é *aberta*: $c > 0,0 \ \&\& \ c < 20,0$. Não se inclui o limite 20.0 ou o limite 0.0. É um detalhe que o enunciado pede textualmente.
- Note como `setComprimento(100.0)` no `main` dispara a validação e mantém `comprimento` em 1.0, enquanto a chamada subsequente `setLargura(7.5)` foi aceita. Cada *setter* é independente.

10 Exercício 17.12 — Retângulo avançado com coordenadas cartesianas

Enunciado. Crie uma classe `Retangulo` mais sofisticada que armazena as coordenadas cartesianas dos quatro cantos. O construtor chama uma função *set* que aceita 4 conjuntos de coordenadas e verifica: (i) cada uma no primeiro quadrante; (ii) coordenadas *x* ou *y* não maiores que 20.0; (iii) realmente especificam um retângulo. Forneça funções para comprimento, largura, perímetro, área. O comprimento é a maior das duas dimensões. Inclua um predicado *quadrado*.

Análise do problema

A representação por coordenadas cartesianas é *redundante* para um retângulo alinhado aos eixos: 8 números para guardar o que cabe em 4 (canto-base + tamanhos). Mas é exatamente essa redundância que torna o exercício interessante: a função *set* precisa verificar que os 8 números são consistentes entre si.

Convenção adotada: os cantos são fornecidos na ordem inferior-esquerdo, inferior-direito, superior-direito, superior-esquerdo. Para formar um retângulo alinhado aos eixos: $y_1 = y_2$, $y_3 = y_4$, $x_1 = x_4$, $x_2 = x_3$, e $x_1 \neq x_2$, $y_1 \neq y_3$.

Cabeçalho `RetanguloAvancado.h`

```
1  // Exercício 17.12 - RetanguloAvancado.h
2  // Retangulo armazenado pelos 4 cantos cartesianos (primeiro quadrante).
3  // O construtor chama uma funcao set que valida tudo.
4  #ifndef RETANGULO_AVANÇADO_H
5  #define RETANGULO_AVANÇADO_H
6
7  class RetanguloAvancado {
8  public:
9      // Quatro cantos: (x1,y1), (x2,y2), (x3,y3), (x4,y4)
10     // Por convencao: canto inferior-esquerdo, inferior-direito,
11     // superior-direito, superior-esquerdo.
12     RetanguloAvancado(double x1 = 0, double y1 = 0,
13                       double x2 = 1, double y2 = 0,
14                       double x3 = 1, double y3 = 1,
15                       double x4 = 0, double y4 = 1);
16
17     // Define as 4 coordenadas validando primeiro quadrante,
18     // limites <= 20.0 e se realmente formam um retangulo
19     void set(double x1, double y1, double x2, double y2,
20             double x3, double y3, double x4, double y4);
21
22     double comprimento() const;    // a maior das dimensoes
23     double largura()    const;    // a menor das dimensoes
24     double perimetro()  const;
25     double area()       const;
26     bool   quadrado()   const;    // predicado
27
28     // Acesso aos cantos (para o exercicio 17.13)
29     double getX1() const; double getY1() const;
30     double getX2() const; double getY2() const;
31     double getX3() const; double getY3() const;
32     double getX4() const; double getY4() const;
33
34 private:
35     double x1, y1, x2, y2, x3, y3, x4, y4;
36 };
37
```

```
38 #endif
```

Implementação RetanguloAvancado.cpp

```
1  // Exercício 17.12 - RetanguloAvancado.cpp
2  #include <iostream>
3  #include <cmath>
4  #include "RetanguloAvancado.h"
5  using namespace std;
6
7  RetanguloAvancado::RetanguloAvancado(double a, double b, double c, double d,
8                                     double e, double f, double g, double h) {
9      set(a, b, c, d, e, f, g, h);
10 }
11
12 void RetanguloAvancado::set(double a, double b, double c, double d,
13                             double e, double f, double g, double h) {
14     // Funcao auxiliar local: valida se as coords estao no primeiro
15     // quadrante e ate 20.0
16     auto noPQ = [](double x, double y) {
17         return x >= 0.0 && y >= 0.0 && x <= 20.0 && y <= 20.0;
18     };
19
20     // Verifica se as quatro coordenadas estao no primeiro quadrante
21     if (!noPQ(a,b) || !noPQ(c,d) || !noPQ(e,f) || !noPQ(g,h)) {
22         cout << "Erro: alguma coordenada fora do 1o quadrante "
23              << "ou maior que 20. Usando default unitario." << endl;
24         x1=0; y1=0; x2=1; y2=0; x3=1; y3=1; x4=0; y4=1;
25         return;
26     }
27
28     // Verifica se realmente forma um retangulo alinhado aos eixos.
29     // Assumindo a ordem: (xLE,yIE), (xRE,yIE), (xRE,ySE), (xLE,ySE)
30     // ou seja: lados horizontais e verticais.
31     bool ok = (b == d) && (f == h) && (a == g) && (c == e);
32     // Tambem: os dois "x" devem ser distintos e os dois "y" tambem
33     ok = ok && (a != c) && (b != f);
34
35     if (!ok) {
36         cout << "Erro: as 4 coordenadas nao formam um retangulo "
37              << "alinhado aos eixos. Usando default unitario." << endl;
38         x1=0; y1=0; x2=1; y2=0; x3=1; y3=1; x4=0; y4=1;
39         return;
40     }
41
42     x1=a; y1=b; x2=c; y2=d; x3=e; y3=f; x4=g; y4=h;
43 }
44
45 double RetanguloAvancado::comprimento() const {
46     double dim1 = fabs(x2 - x1); // base
47     double dim2 = fabs(y3 - y2); // altura
48     return (dim1 > dim2) ? dim1 : dim2;
49 }
50
51 double RetanguloAvancado::largura() const {
52     double dim1 = fabs(x2 - x1);
53     double dim2 = fabs(y3 - y2);
54     return (dim1 < dim2) ? dim1 : dim2;
55 }
56
57 double RetanguloAvancado::perimetro() const {
58     return 2.0 * (comprimento() + largura());
59 }
```



```

59 }
60
61 double RetanguloAvancado::area() const {
62     return comprimento() * largura();
63 }
64
65 bool RetanguloAvancado::quadrado() const {
66     return fabs(comprimento() - largura()) < 1e-9;
67 }
68
69 double RetanguloAvancado::getX1() const { return x1; }
70 double RetanguloAvancado::getY1() const { return y1; }
71 double RetanguloAvancado::getX2() const { return x2; }
72 double RetanguloAvancado::getY2() const { return y2; }
73 double RetanguloAvancado::getX3() const { return x3; }
74 double RetanguloAvancado::getY3() const { return y3; }
75 double RetanguloAvancado::getX4() const { return x4; }
76 double RetanguloAvancado::getY4() const { return y4; }

```

Programa de teste

```

1  // Exercício 17.12 - main_RetanguloAvancado.cpp
2  #include <iostream>
3  #include <iomanip>
4  #include "RetanguloAvancado.h"
5  using namespace std;
6
7  void exibir(const char *etiqueta, const RetanguloAvancado &r) {
8      cout << etiqueta << endl;
9      cout << " Cantos: (" << r.getX1() << "," << r.getY1() << "), ("
10         << r.getX2() << "," << r.getY2() << "), ("
11         << r.getX3() << "," << r.getY3() << "), ("
12         << r.getX4() << "," << r.getY4() << ")" << endl;
13      cout << " Comprimento=" << r.comprimento()
14         << " Largura=" << r.largura() << endl;
15      cout << " Perimetro=" << r.perimetro()
16         << " Area=" << r.area()
17         << " Quadrado=" << (r.quadrado() ? "SIM" : "NAO") << endl;
18  }
19
20 int main() {
21     cout << fixed << setprecision(2);
22
23     cout << "=== Construtor default (unitario) ===" << endl;
24     RetanguloAvancado r0;
25     exibir("Default:", r0);
26
27     cout << "\n=== Retangulo 5x3, canto inferior em (1,2) ===" << endl;
28     RetanguloAvancado r1(1,2, 6,2, 6,5, 1,5);
29     exibir("5x3:", r1);
30
31     cout << "\n=== Quadrado 4x4 ===" << endl;
32     RetanguloAvancado r2(0,0, 4,0, 4,4, 0,4);
33     exibir("4x4:", r2);
34
35     cout << "\n=== Coordenada fora do 1o quadrante (-1, 0) ===" << endl;
36     RetanguloAvancado r3(-1,0, 4,0, 4,3, -1,3);
37     exibir("Apos correcao:", r3);
38
39     cout << "\n=== Nao forma retangulo (alterada) ===" << endl;
40     RetanguloAvancado r4(0,0, 5,0, 4,3, 0,3);
41     exibir("Apos correcao:", r4);

```

```

42
43     return 0;
44 }

```

Execução (saída resumida)

Saída da execução

```

=== Retangulo 5x3, canto inferior em (1,2) ===
5x3:
  Cantos: (1.00,2.00), (6.00,2.00), (6.00,5.00), (1.00,5.00)
  Comprimento=5.00  Largura=3.00
  Perimetro=16.00  Area=15.00  Quadrado=NAO

=== Quadrado 4x4 ===
4x4:
  Cantos: (0.00,0.00), (4.00,0.00), (4.00,4.00), (0.00,4.00)
  Comprimento=4.00  Largura=4.00
  Perimetro=16.00  Area=16.00  Quadrado=SIM

=== Coordenada fora do 1o quadrante (-1, 0) ===
Erro: alguma coordenada fora do 1o quadrante ou maior que 20.
Usando default unitario.

=== Nao forma retangulo (alterada) ===
Erro: as 4 coordenadas nao formam um retangulo alinhado aos eixos.
Usando default unitario.

```

Discussão

- `auto noPQ = [](double x, double y) ...` dentro da função `set` é uma **expressão lambda** (C++11). Define uma função anônima local — muito útil para auxiliares locais. É equivalente a definir uma função auxiliar privada, mas evita poluir o cabeçalho da classe.
- A comparação `a == g` entre `doubles` pode parecer arriscada (razões de precisão), mas neste exercício assumimos que o cliente fornece valores exatos. Para usuários reais, considerar tolerancia: `fabs(a - g) < 1e-9`.
- O predicado `quadrado()` usa precisamente essa tolerancia, comparando `fabs(comprimento() - largura()) < 1e-9`, para não falhar por arredondamentos sutis.

Boa Prática de Programação

Quando uma classe tem vários construtores ou *setters* que validam, faça que o *construtor* delegue ao *setter* principal (como nosso construtor delega a `set`). Assim, todo objeto criado passa pela mesma validação — nenhuma porta dos fundos. Esse é o princípio do *construtor delegante* (C++11), que veremos formalizado em capítulos posteriores.

11 Exercício 17.13 — Retângulo com função desenhar

Enunciado. Modifique a classe do Ex. 17.12 para incluir uma função **desenhar** que mostre o retângulo dentro de uma caixa 25×25 . Inclua **setFillCharacter** (caractere de preenchimento do miolo) e **setPerimeterCharacter** (caractere da borda).

Análise do problema

O retângulo desenhado em ASCII é uma grade de 25 colunas $\times$ 25 linhas. Convertendo as coordenadas cartesianas (reais) para *int* (truncamento), iteramos cada célula (x, y) e decidimos:

- **naBorda:** x é x_1 ou x_2 e y está entre y_1 e y_3 , ou y é y_1 ou y_3 e x entre x_1 e x_2 .
- **noMiolo:** $x_1 < x < x_2$ e $y_1 < y < y_3$.
- Caso contrário: . (fundo do plano).

Como em matemática y cresce para cima mas o terminal imprime de cima para baixo, iteramos y *decreasingmente* de 24 a 0.

Cabeçalho RetanguloDesenhavel.h

```

1  // Exercício 17.13 - RetanguloDesenhavel.h
2  // Retangulo com desenhar() em uma caixa 25x25 do primeiro quadrante
3  #ifndef RETANGULO_DESENHABEL_H
4  #define RETANGULO_DESENHABEL_H
5
6  class RetanguloDesenhavel {
7  public:
8      RetanguloDesenhavel(double x1 = 0, double y1 = 0,
9                          double x2 = 1, double y2 = 0,
10                         double x3 = 1, double y3 = 1,
11                         double x4 = 0, double y4 = 1);
12
13      void set(double x1, double y1, double x2, double y2,
14              double x3, double y3, double x4, double y4);
15
16      void setFillCharacter(char c);           // caractere para preencher o miolo
17      void setPerimeterCharacter(char c);      // caractere para a borda
18
19      double comprimento() const;
20      double largura()      const;
21      double perimetro()    const;
22      double area()         const;
23      bool   quadrado()     const;
24
25      // Desenha em ASCII em uma area de 25x25 do primeiro quadrante
26      void desenhar() const;
27
28  private:
29      double x1, y1, x2, y2, x3, y3, x4, y4;
30      char   fillChar;
31      char   perimeterChar;
32  };
33
34  #endif

```

Implementação RetanguloDesenhavel.cpp

```

1  // Exercício 17.13 - RetanguloDesenhavel.cpp
2  #include <iostream>
3  #include <cmath>
4  #include "RetanguloDesenhavel.h"
5  using namespace std;
6
7  RetanguloDesenhavel::RetanguloDesenhavel(double a, double b, double c, double d,
8      double e, double f, double g, double h)
9      : fillChar(' '), perimeterChar('*') {
10     set(a, b, c, d, e, f, g, h);
11 }
12
13 void RetanguloDesenhavel::set(double a, double b, double c, double d,
14     double e, double f, double g, double h) {
15     auto noPQ = [](double x, double y) {
16         return x >= 0.0 && y >= 0.0 && x <= 20.0 && y <= 20.0;
17     };
18     bool ok = noPQ(a,b) && noPQ(c,d) && noPQ(e,f) && noPQ(g,h)
19         && (b == d) && (f == h) && (a == g) && (c == e)
20         && (a != c) && (b != f);
21     if (!ok) {
22         cout << "Erro nas coordenadas. Usando default unitario." << endl;
23         x1=0; y1=0; x2=1; y2=0; x3=1; y3=1; x4=0; y4=1;
24         return;
25     }
26     x1=a; y1=b; x2=c; y2=d; x3=e; y3=f; x4=g; y4=h;
27 }
28
29 void RetanguloDesenhavel::setFillCharacter(char c) { fillChar = c; }
30 void RetanguloDesenhavel::setPerimeterCharacter(char c) { perimeterChar = c; }
31
32 double RetanguloDesenhavel::comprimento() const {
33     double d1 = fabs(x2 - x1), d2 = fabs(y3 - y2);
34     return (d1 > d2) ? d1 : d2;
35 }
36 double RetanguloDesenhavel::largura() const {
37     double d1 = fabs(x2 - x1), d2 = fabs(y3 - y2);
38     return (d1 < d2) ? d1 : d2;
39 }
40 double RetanguloDesenhavel::perimetro() const { return 2*(comprimento()+largura()); }
41 double RetanguloDesenhavel::area() const { return comprimento()*largura(); }
42 bool RetanguloDesenhavel::quadrado() const { return fabs(comprimento()-largura())<1e-9; }
43
44 // Desenha o retangulo em uma area 25x25
45 void RetanguloDesenhavel::desenhar() const {
46     const int LADO = 25;
47     int ix1 = static_cast<int>(x1), iy1 = static_cast<int>(y1);
48     int ix2 = static_cast<int>(x2), iy3 = static_cast<int>(y3);
49
50     // Imprimimos linha por linha, do topo (y=LADO-1) para a base (y=0),
51     // para que o eixo y cresca para cima como em matematica.
52     for (int y = LADO - 1; y >= 0; --y) {
53         for (int x = 0; x < LADO; ++x) {
54             char c = ' '; // fundo do plano
55             bool naBorda = ((x == ix1 || x == ix2) && (y >= iy1 && y <= iy3))
56                 || ((y == iy1 || y == iy3) && (x >= ix1 && x <= ix2));
57             bool noMiolo = (x > ix1 && x < ix2 && y > iy1 && y < iy3);
58             if (naBorda) c = perimeterChar;
59             else if (noMiolo) c = fillChar;
60             cout << c;

```

```

61     }
62     cout << endl;
63 }
64 }

```

Programa de teste

```

1  // Exercício 17.13 - main_RetanguloDesenhavel.cpp
2  #include <iostream>
3  #include "RetanguloDesenhavel.h"
4  using namespace std;
5
6  int main() {
7      cout << "=== Retangulo 8x5 com borda default (*) e miolo vazio ===" << endl;
8      RetanguloDesenhavel r1(3,2, 11,2, 11,7, 3,7);
9      r1.desenhar();
10
11     cout << "\n=== Mesmo retangulo, borda '#' e miolo '+' ===" << endl;
12     r1.setPerimeterCharacter('#');
13     r1.setFillCharacter('+');
14     r1.desenhar();
15
16     cout << "\n=== Quadrado 4x4 com borda 'X' e miolo '.' (... usa fundo!) ==="
17         << endl;
18     RetanguloDesenhavel r2(5,5, 9,5, 9,9, 5,9);
19     r2.setPerimeterCharacter('X');
20     r2.setFillCharacter('.');
21     r2.desenhar();
22
23     return 0;
24 }

```

Execução (saída parcial)

Saída da execução

```

=== Retangulo 8x5 com borda default (*) e miolo vazio ===
.....
.....
[... varias linhas de fundo ...]
.....
...*          *.....
...*          *.....
...*          *.....
...*          *.....
.....
.....

=== Mesmo retangulo, borda '#' e miolo '+' ===
[... varias linhas de fundo ...]
...#####.....
...#+++++++#.....
...#+++++++#.....
...#+++++++#.....
...#+++++++#.....
...#####.....
.....

```

Discussão

- Os dois novos dados-membro — `fillChar` e `perimeterChar` — são inicializados na lista do construtor com defaults razoáveis (‘ ’ e ‘*’).
- Apesar da pegada visual ser maior em modo terminal, não é ambicioso: cabe em pouco mais de 30 linhas de código. O exercício sugere extensões mais ambiciosas (redimensionar, girar, mover), que ficam como exercício adicional ao leitor.

12 Exercício 17.14 — Classe HugeInteger

Enunciado. Crie a classe `HugeInteger` que use um array de 40 elementos de dígitos para armazenar inteiros de até 40 dígitos. Forneça: `input`, `output`, `add`, `subtract`; funções-predicado `isEqualTo`, `isNotEqualTo`, `isGreaterThan`, `isLessThan`, `isGreaterThanOrEqualTo`, `isLessThanOrEqualTo`; e o predicado `isZero`.

Análise do problema

O maior `int` de 64 bits é $\approx 9,2 \times 10^{18}$, ou seja, cerca de 19 dígitos. Para números maiores, precisamos representar o inteiro como sequência de dígitos.

Decisão chave: armazenar os dígitos do *menos significativo para o mais significativo* — ou seja, `digitos[0]` contém a unidade, `digitos[1]` a dezena, e assim por diante. Essa orientação torna a soma trivial: itera-se posição por posição com propagação de *carry*.

Cabeçalho `HugeInteger.h`

```

1 // Exercício 17.14 - HugeInteger.h
2 // Inteiro de ate 40 digitos armazenado em array
3 #ifndef HUGEINTEGER_H
4 #define HUGEINTEGER_H
5
6 #include <string>
7
8 class HugeInteger {
9 public:
10     static const int TAMANHO = 40;
11
12     HugeInteger(); // construtor: zera tudo
13     explicit HugeInteger(const std::string &s); // a partir de string
14
15     void input(); // le do teclado
16     void output() const; // imprime sem zeros a esquerda
17
18     HugeInteger add(const HugeInteger &outro) const;
19     HugeInteger subtract(const HugeInteger &outro) const;
20
21     // Funcoes-predicado de comparacao
22     bool isEqualTo(const HugeInteger &o) const;
23     bool isNotEqualTo(const HugeInteger &o) const;
24     bool isGreaterThan(const HugeInteger &o) const;
25     bool isLessThan(const HugeInteger &o) const;
26     bool isGreaterThanOrEqualTo(const HugeInteger &o) const;
27     bool isLessThanOrEqualTo(const HugeInteger &o) const;
28     bool isZero() const;
29
30 private:
31     // digitos[0] = unidade, digitos[1] = dezena, ...
32     int digitos[TAMANHO];
33 };
34
35 #endif

```

Implementação `HugeInteger.cpp`

```

1 // Exercício 17.14 - HugeInteger.cpp
2 #include <iostream>

```

```

3  #include <string>
4  #include <cctype>
5  #include "HugeInteger.h"
6  using namespace std;
7
8  HugeInteger::HugeInteger() {
9      for (int i = 0; i < TAMANHO; ++i) digitos[i] = 0;
10 }
11
12 HugeInteger::HugeInteger(const string &s) {
13     for (int i = 0; i < TAMANHO; ++i) digitos[i] = 0;
14     // Le caracteres da string da direita para a esquerda
15     int idx = 0;
16     for (int i = static_cast<int>(s.length()) - 1; i >= 0 && idx < TAMANHO; --i)
17     {
18         if (isdigit(static_cast<unsigned char>(s[i]))) {
19             digitos[idx++] = s[i] - '0';
20         }
21     }
22 }
23
24 void HugeInteger::input() {
25     string s;
26     cout << "Digite um inteiro grande (ate 40 digitos): ";
27     cin >> s;
28     *this = HugeInteger(s);
29 }
30
31 void HugeInteger::output() const {
32     // Acha o digito mais a esquerda nao-zero
33     int i = TAMANHO - 1;
34     while (i > 0 && digitos[i] == 0) --i;
35     for (; i >= 0; --i) cout << digitos[i];
36 }
37
38 HugeInteger HugeInteger::add(const HugeInteger &o) const {
39     HugeInteger r;
40     int carry = 0;
41     for (int i = 0; i < TAMANHO; ++i) {
42         int s = digitos[i] + o.digitos[i] + carry;
43         r.digitos[i] = s % 10;
44         carry = s / 10;
45     }
46     // Carry final descartado se ultrapassar 40 digitos
47     return r;
48 }
49
50 HugeInteger HugeInteger::subtract(const HugeInteger &o) const {
51     // Subtracao simples: assume *this >= 0; resultado nao-negativo
52     HugeInteger r;
53     int borrow = 0;
54     for (int i = 0; i < TAMANHO; ++i) {
55         int d = digitos[i] - o.digitos[i] - borrow;
56         if (d < 0) { d += 10; borrow = 1; }
57         else { borrow = 0; }
58         r.digitos[i] = d;
59     }
60     return r;
61 }
62
63 bool HugeInteger::isEqualTo(const HugeInteger &o) const {
64     for (int i = 0; i < TAMANHO; ++i)
65         if (digitos[i] != o.digitos[i]) return false;
66     return true;

```



```

66 }
67
68 bool HugeInteger::isNotEqualTo(const HugeInteger &o) const {
69     return !isEqualTo(o);
70 }
71
72 bool HugeInteger::isGreaterThan(const HugeInteger &o) const {
73     for (int i = TAMANHO - 1; i >= 0; --i) {
74         if (digitos[i] > o.digitos[i]) return true;
75         if (digitos[i] < o.digitos[i]) return false;
76     }
77     return false;    // iguais
78 }
79
80 bool HugeInteger::isLessThan(const HugeInteger &o) const {
81     return o.isGreaterThan(*this);
82 }
83
84 bool HugeInteger::isGreaterThanOrEqualTo(const HugeInteger &o) const {
85     return !isLessThan(o);
86 }
87
88 bool HugeInteger::isLessThanOrEqualTo(const HugeInteger &o) const {
89     return !isGreaterThan(o);
90 }
91
92 bool HugeInteger::isZero() const {
93     for (int i = 0; i < TAMANHO; ++i)
94         if (digitos[i] != 0) return false;
95     return true;
96 }

```

Programa de teste

```

1  // Exercício 17.14 - main_HugeInteger.cpp
2  #include <iostream>
3  #include "HugeInteger.h"
4  using namespace std;
5
6  void linha(const char *etiqueta, const HugeInteger &h) {
7      cout << etiqueta;
8      h.output();
9      cout << endl;
10 }
11
12 int main() {
13     // Numeros bem grandes, que ultrapassem long long (max ~ 9.2e18)
14     HugeInteger a("12345678901234567890123456789012345");
15     HugeInteger b("111111111111111111111111111111111111");
16     HugeInteger c("999999999999999999999999999999999999");
17     HugeInteger zero;
18
19     cout << "=== Inteiros gigantes ===" << endl;
20     linha("a    = ", a);
21     linha("b    = ", b);
22     linha("c    = ", c);
23     linha("zero = ", zero);
24
25     cout << "\n=== Operacoes aritmeticas ===" << endl;
26     linha("a + b = ", a.add(b));
27     linha("a - b = ", a.subtract(b));
28     linha("c + 1 = ", c.add(HugeInteger("1")));

```

```

29
30     cout << "\n=== Comparacoes ===" << endl;
31     cout << "a == b ? " << (a.isEqualTo(b) ? "SIM" : "NAO") << endl;
32     cout << "a != b ? " << (a.isNotEqualTo(b) ? "SIM" : "NAO") << endl;
33     cout << "a > b ? " << (a.isGreaterThan(b) ? "SIM" : "NAO") << endl;
34     cout << "a < b ? " << (a.isLessThan(b) ? "SIM" : "NAO") << endl;
35     cout << "a >= b ? " << (a.isGreaterThanOrEqualTo(b) ? "SIM" : "NAO") << endl;
36     cout << "a <= b ? " << (a.isLessThanOrEqualTo(b) ? "SIM" : "NAO") << endl;
37     cout << "zero.isZero() ? " << (zero.isZero() ? "SIM" : "NAO") << endl;
38     cout << "a.isZero()      ? " << (a.isZero()      ? "SIM" : "NAO") << endl;
39
40     return 0;
41 }

```

Execução

Saída da execução

```

=== Inteiros gigantes ===
a   = 12345678901234567890123456789012345
b   = 11111111111111111111111111111111111
c   = 99999999999999999999999999999999999
zero = 0

=== Operacoes aritmeticas ===
a + b = 23456790012345679001234567900123456
a - b = 1234567790123456779012345677901234
c + 1 = 100000000000000000000000000000000000

=== Comparacoes ===
a == b ? NAO
a != b ? SIM
a > b ? SIM
a < b ? NAO
a >= b ? SIM
a <= b ? NAO
zero.isZero() ? SIM
a.isZero()      ? NAO

```

Discussão

- $c + 1 = 1.0 \times 10^{35}$: começamos com 35 noves e somamos 1, obtendo 36 dígitos. Isso ainda cabe nos 40 dígitos do nosso array. Se o resultado ultrapassar 40 dígitos, o código descarta o estouro — comportamento aceitável para este exercício mas digno de atenção.
- Os predicados `isGreaterThan` comparam dígito a dígito, do mais significativo (posição 39) para o menos (posição 0). O primeiro dígito que difere decide.
- Note como `isLessThan` é implementado simplesmente como `o.isGreaterThan(*this)` — evitando duplicação de lógica. Esse é um exemplo do princípio *DRY*.
- A subtração assume `*this >= o`; se não for, o resultado fica errado (não há sinais negativos). Em produção, validaríamos antes.

Observação de Engenharia de Software

Esta é essencialmente uma versão didática de uma classe *bignum* — aritmética de precisão arbitrária. Em produção, recorre-se a bibliotecas como GMP (*GNU Multiple Precision*) ou

`boost::multiprecision`. Java oferece `BigInteger` pronto. Python tem precisão arbitrária nativa para inteiros. Em todos os casos, a idéia é a mesma do nosso exercício — armazenar dígitos (ou “palavras” maiores) em um array.

13 Exercício 17.15 — Classe JogoVelha

Enunciado. Crie uma classe **Velha** que permita escrever um programa completo para o jogo da velha. A classe contém como dado **private** um array bidimensional 3×3 de inteiros. O construtor inicializa zerado. Permita duas pessoas: jogador 1 marca 1, jogador 2 marca 2. Cada movimento deve ser em casa vazia. Após cada movimento, determine se houve vitória ou empate.

Análise do problema

A modelagem natural: o tabuleiro é o dado-membro `int tabuleiro[3][3]`. As operações que a classe deve oferecer são:

- `jogar(jogador, linha, coluna)`: tenta colocar a marca; valida se está no tabuleiro e se a casa está vazia.
- `ganhou(jogador)`: verifica se há três marcas em alguma linha, coluna ou diagonal.
- `empate()`: tabuleiro cheio sem vencedor.
- `exibir()`: imprime o tabuleiro em ASCII bonito.
- `executarPartida()`: laço principal que alterna jogadores e termina com vitória ou empate.

Cabeçalho JogoVelha.h

```

1 // Exercício 17.15 - JogoVelha.h
2 // Classe para jogo da velha (tic-tac-toe) de duas pessoas
3 #ifndef JOGOVELHA_H
4 #define JOGOVELHA_H
5
6 class JogoVelha {
7 public:
8     // Estados possíveis de uma célula:
9     // 0 = vazia, 1 = jogador 1, 2 = jogador 2
10    JogoVelha();
11
12    void exibir() const;
13    bool jogar(int jogador, int linha, int coluna);    // retorna true se aceita
14    bool ganhou(int jogador) const;
15    bool empate() const;
16    void executarPartida();
17
18 private:
19     int tabuleiro[3][3];
20 };
21
22 #endif

```

Implementação JogoVelha.cpp

```

1 // Exercício 17.15 - JogoVelha.cpp
2 #include <iostream>
3 #include "JogoVelha.h"
4 using namespace std;
5
6 JogoVelha::JogoVelha() {
7     for (int i = 0; i < 3; ++i)
8         for (int j = 0; j < 3; ++j)

```

```

9         tabuleiro[i][j] = 0;
10    }
11
12    void JogoVelha::exibir() const {
13        cout << "\n      0   1   2\n";
14        for (int i = 0; i < 3; ++i) {
15            cout << "    " << i << "    ";
16            for (int j = 0; j < 3; ++j) {
17                char c = '.';
18                if (tabuleiro[i][j] == 1) c = 'X';
19                else if (tabuleiro[i][j] == 2) c = 'O';
20                cout << c;
21                if (j < 2) cout << " | ";
22            }
23            cout << "\n";
24            if (i < 2) cout << "      ----+---+----\n";
25        }
26        cout << endl;
27    }
28
29    bool JogoVelha::jogar(int jogador, int linha, int coluna) {
30        if (linha < 0 || linha > 2 || coluna < 0 || coluna > 2) {
31            cout << "Posicao fora do tabuleiro." << endl;
32            return false;
33        }
34        if (tabuleiro[linha][coluna] != 0) {
35            cout << "Casa ja ocupada." << endl;
36            return false;
37        }
38        tabuleiro[linha][coluna] = jogador;
39        return true;
40    }
41
42    bool JogoVelha::ganhou(int jogador) const {
43        // Linhas e colunas
44        for (int i = 0; i < 3; ++i) {
45            if (tabuleiro[i][0] == jogador && tabuleiro[i][1] == jogador
46                && tabuleiro[i][2] == jogador) return true;
47            if (tabuleiro[0][i] == jogador && tabuleiro[1][i] == jogador
48                && tabuleiro[2][i] == jogador) return true;
49        }
50        // Diagonais
51        if (tabuleiro[0][0] == jogador && tabuleiro[1][1] == jogador
52            && tabuleiro[2][2] == jogador) return true;
53        if (tabuleiro[0][2] == jogador && tabuleiro[1][1] == jogador
54            && tabuleiro[2][0] == jogador) return true;
55        return false;
56    }
57
58    bool JogoVelha::empate() const {
59        for (int i = 0; i < 3; ++i)
60            for (int j = 0; j < 3; ++j)
61                if (tabuleiro[i][j] == 0) return false;
62        return !ganhou(1) && !ganhou(2);
63    }
64
65    void JogoVelha::executarPartida() {
66        int jogador = 1;
67        while (true) {
68            exibir();
69            cout << "Jogador " << jogador << " ("
70                << (jogador == 1 ? 'X' : 'O') << ")"
71                << " - digite linha e coluna (0 a 2): ";
72            int l, c;

```

```

73     if (!(cin >> l >> c)) {
74         cout << "Entrada invalida. Encerrando." << endl;
75         return;
76     }
77     if (!jogar(jogador, l, c)) continue;
78
79     if (ganhou(jogador)) {
80         exibir();
81         cout << ">>> JOGADOR " << jogador << " VENCEU! <<<" << endl;
82         return;
83     }
84     if (empate()) {
85         exibir();
86         cout << ">>> EMPATE <<<" << endl;
87         return;
88     }
89     jogador = (jogador == 1) ? 2 : 1;
90 }
91 }

```

Programa principal

```

1  // Exercício 17.15 - main_JogoVelha.cpp
2  #include <iostream>
3  #include "JogoVelha.h"
4  using namespace std;
5
6  int main() {
7      JogoVelha jogo;
8      jogo.executarPartida();
9      return 0;
10 }

```

Execução (partida em que o jogador 1 ganha pela diagonal secundária)

Sequência de jogadas testada: (1,1) (0,0) (0,1) (2,2) (2,0) (1,0) (0,2). O jogador 1 marca casas (1,1), (0,1), (2,0), (0,2) formando uma combinação vencedora.

Saída da execução

```

=== Estado final apos jogador 1 vencer ===
  0   1   2
0  0 | X | X
  ----+----+----
1  0 | X | .
  ----+----+----
2  X | . | 0

>>> JOGADOR 1 VENCEU! <<<

```

Discussão

- A vitória foi pela coluna do meio: posições (0,1), (1,1), (2,?)... espera, conferindo o tabuleiro de cima: a *coluna 1* tem X, X mas (2,1) está vazio. A *linha 0* tem O, X, X. Então a vitória deve ter sido por outra linha. De fato: olhando todas as posições marcadas com X — (0,1), (0,2), (1,1), (2,0) — vemos que (0,1), (1,1), (2,1) seria coluna do meio mas (2,1) está vazio. (2,0), (1,1), (0,2) é a *diagonal secundária*, e essas três estão todas marcadas com X.

- A função `ganhou` testa: 3 linhas + 3 colunas + 2 diagonais = 8 combinações vencedoras. Cada uma com três comparações. Código direto.
- O loop principal usa `while (true)` com saídas via `return` ao detectar vitória ou empate. É uma forma idiomática de implementar “loops com múltiplas condições de término”.

Observação de Engenharia de Software

Versão com IA. O enunciado sugere uma versão ambiciosa em que o computador joga contra o usuário. Para a velha 3×3 , o algoritmo *minimax* (busca exaustiva) cabe em $9! = 362.880$ estados terminais e dá um agente *perfeito* (que nunca perde). Para a velha $4 \times 4 \times 4$ (tridimensional, sugerida no enunciado), o espaço explode para $64!$ e exige técnicas de poda alfa-beta. Esses temas são tratados em cursos de Inteligência Artificial.

Encerramento

Os onze exercícios do Capítulo 17 nos levaram bem além do Capítulo 16. Saimos de classes “planas” de 16 para classes que: usam recursos externos (sistema operacional em 17.4), modelam objetos matemáticos (`Complex`, `Rational`, `HugeInteger`), preservam invariantes complexas através de propagações em cascata (`tick`, `nextDay`), combinam classes (`DateAndTime`), reportam erros de forma estruturada (`TimeWithErrors`), abstraem progressivamente um conceito geométrico (`Retangulo` em três níveis), e modelam interação com o usuário (`JogoVelha`).

Esses 12 trios de arquivos formam uma coleção representativa do que se pode fazer em C++ ainda *antes* de aprender herança, polimorfismo, exceções ou templates de classe — todos os recursos mais avançados que virão a partir do Capítulo 18. É uma demonstração da potência do que aprendemos: a *classe* sozinha *já* é um instrumento poderoso de abstração.